

Problem statement 1: Employee salary system

- create program base class Employee with virtual salary()
- derive full time and intern and override it
- use Base pointer to call salary

Problem statement 2: Shape Area.

- create base class shape with virtual area()
- derive Circle and Rectangle and override it
- use Base pointer to find correct area

Problem statement 3: Animal sound.

- Create base class Animal with virtual sound()
- derive Dog and Cat and override it
- store object in base pointer array and call sound - C uses loop.

```
int main() {
    Shape * ptr = new Rectangle();
    ptr->area(); // Base version called.
}
```

using virtual

ptr->area(); → rectangle version called

Key concept

without virtual → compile time binding

with virtual → run time binding (late binding)

Rules

- Must be declared in base class
- usually overridden in derived class
- work with base class pointer.

Real meaning

Shape * ptr = new Rectangle();

ptr type → Shape*

Object type → Rectangle.

with virtual → function based on object
 without virtual → " " " " ~~base~~ pointer

Why is it important

- used in polymorphism
- helps in writing flexible & reusable code.
- very important for interview.

Problem statement 2: Person and Student

- Base $\rightarrow$ Person $\rightarrow$ store name
- derived $\rightarrow$ student $\rightarrow$ store mark

Tasks : Same. But using Dynamically.

Problem statement 3: Shape and Rectangle.

- Base $\rightarrow$ ~~Area~~ shape $\rightarrow$ Area() & "finding area"
- derived $\rightarrow$ rectangle $\rightarrow$ length + breadth and
Calculation.

Day-43 (Virtual function)

A virtual function is a function in a base class that is over-ridden in the derived class and is called based on the object type at run time.

without virtual function.

```
class shape {
```

```
public :
```

```
virtual void area() {
```

```
cout << "Base area" << endl;
```

```
}
```

```
};
```

```
class Rectangle : public shape {
```

```
public :
```

```
void area() {
```

```
cout << "Rectangle area" << endl;
```

```
};
```